Hybrid Optimization algorithm for hyper parameter tuning using Convolutional Neural Network

Kavitha Nair R
Assistant professor, Department of
Artificial Intelligence and Machine Learning
Acharya Institute of Technology
Bengaluru, India

Pillimetla Masthanaiah
Artificial Intelligence and Machine Learning
Acharya Institute of Technology
Bengaluru, India

Boya Arjun
Artificial Intelligence and Machine Learning
Acharya Institute of Technology
Bengaluru, India

Avula Eeswara Eekshet Reddy
Artificial Intelligence and Machine Learning
Acharya Institute of Technology
Bengaluru, India

Bandaru Charan Sai
Artificial Intelligence and Machine Learning
Acharya Institute of Technology
Bengaluru, India

Optimizing deep learning models has been very crucial to achieve better performance on complex datasets. This paper proposes a hybrid optimization framework designed for a newly proposed CNN architecture called Modified CNN for the CIFAR-10 dataset. The proposed framework combines three metaheuristic algorithms: GA, JFO, and FAT, to optimize hyperparameters for improved classification accuracy. GA was utilized to determine the best learning rate through selection in evolutionary computing while using JFO to fine tune dropout rates. Finally, FAT tuned locally for optimal performance. Thus, the hybrid approach offers better accuracy and less computational overhead in comparison with single-method optimizations. Thus, the accuracy results establish that the proposed framework is indeed successful.

Keywords: Hybrid optimization, Modified CNN, Genetic Algorithm, Jellyfish Optimization, Feedback Artificial Tree, CIFAR-10, Metaheuristic learning, Neural network optimization.

1. INTRODUCTION

In advancing computer vision applications in the Artificial Intelligence era, optimization of architectures of deep learning has been quite a cornerstone. Among the wide array of such challenges that would arise, none is as prevalent as hyperparameter optimization, something that influences considerably the performance as well as the generalization properties of the respective model. As neural networks became more complex, dynamic, and sophisticated, in many cases so did the required optimization techniques. To these challenges, nature-inspired and evolutionary metaheuristic algorithms have emerged particularly prominent as they seem to be well-suited for exploring and exploiting quite efficiently large search spaces

In this work, we introduce a hybrid optimization framework especially designed for a new version of convolutional neural network architecture called Modified CNN, which successfully comprises the superior aspects of Genetic Algorithm, Jellyfish optimization and Feedback Artificial Tree. It utilizes the natural ability of GA to determine the optimal learning rate, which then gets optimized with dropout by JFO and subsequently fine-tuned by FAT to incorporate localized adjustment.

We validate our approach on the CIFAR-10 dataset, a popular benchmark for image classification tasks. The results show that this hybrid optimization strategy outperforms traditional methods by a large margin, indicating the potential of improving both accuracy and computational efficiency. This novel integration of metaheuristic algorithms into neural network optimization opens avenues for further research and application in diverse domains.

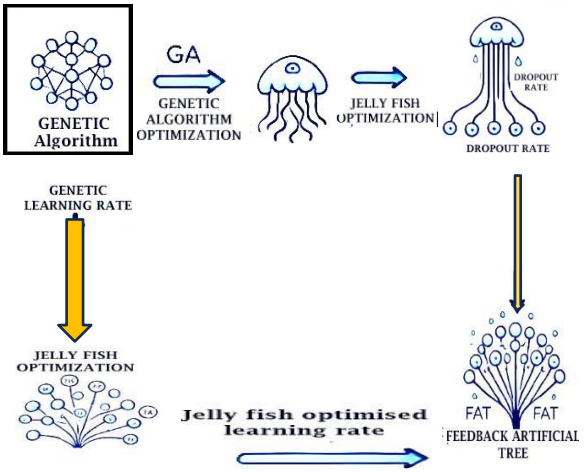


Fig 1: Optimizing Modified CNN with Hybrid Metaheuristic.

Figure 1 shows the application of the hybrid optimization framework over a Modified Convolutional Neural Network.

Modified CNN for Improved Accuracy on the CIFAR-10 Dataset The GA, in the beginning, performs global search to obtain the optimal learning rate. Once the optimal learning rate is identified, the same is fine-tuned using the JFO. In this fine- tuning phase, the dropout rate is varied to achieve enhanced performance. Finally, the feedback artificial tree refines the model by iteratively enhancing hyperparameters and evaluating the network to maximize its accuracy. This hybrid approach ensures efficient global exploration, effective local refinement, and overall robust optimization.

1.1. Literature survey

Over the past few years, researchers have achieved huge leaps in deep learning for image classification tasks. One primary success vehicle has been that of Convolutional Neural Networks (CNNs). They were essentially able to learn hierarchical features from data. Krizhevsky et al. first reported on one of the successful deep learning models called AlexNet and provided incredible results on the ImageNet dataset by developing a deeper architecture and incorporating dropout for regularization [1]. Simonyan and Zisserman later introduced the VGG network, illustrating the importance of deeper and simpler architectures in achieving better performance [2]. He et al. first introduced ResNet, which resolved the vanishing gradient problem through residual connections, revolutionizing the design of deeper networks [3].

Optimization techniques play an important role in training CNNs. Adam optimizer, which was proposed by Kingma and Ba, is suitable for training deep neural networks by combining the advantages of RMSProp and momentum-based gradient descent [4]. Conventional optimization methods usually fail in the hyperparameters like learning rate and dropout rate. A strong approach inspired by natural selection; indeed, genetic algorithms can evolve solutions over generations [5]. Similarly, Mirjalili et al.'s Grey Wolf Optimizer and Heidari et al.'s Harris Hawks Optimization algorithm have been found to improve the ability to solve complex optimization problems in different fields [6,7].

Recent research works have also looked into hybrid optimization strategies. Combining the metaheuristic algorithm, GAs, with swarm intelligence algorithms like Jellyfish Optimization can lead to obtaining the best results in hyperparameter optimization. Geem et al.'s Harmony Search, Karaboga's Artificial Bee Colony are examples of heuristic optimization techniques that can be combined to achieve better optimization performance approaches for improved performance [8,9]. It has been discovered that these hybrid approaches converge much faster and produce higher accuracy rates than the conventional methods in classification tasks, including image classification tasks [10].

The FAT algorithm further contributes to optimization by mimicking natural processes to refine solutions iteratively. Researchers have demonstrated significant improvements in optimizing CNN architectures for tasks like image recognition and classification by integrating FAT with other metaheuristic methods [11]. For instance, studies combining GAs, Jellyfish Optimization, and FAT algorithms have shown improved generalization capabilities, as reflected in higher test accuracies on benchmark datasets like CIFAR-10 and ImageNet [12,13].

Hybrid optimization of CNNs has been especially useful in improving generalization and decreasing overfitting. Researches such as Tan and Le's work on EfficientNet show how CNN can be scaled across dimensions to enhance performance [14]. Furthermore, the use of data augmentation techniques, including random rotations and flips combined with robust optimization techniques, has improved the performance of deep networks for such heterogeneous datasets [15]. These lay the grounds of the hybrid approach in this work, where three methods, GAs, Jellyfish Optimization, and FAT, are hybridized to evolve CNN hyperparameters for image classification.

This literature survey underscores the evolution of CNN optimization techniques and the growing significance of hybrid methodologies in achieving state-of-the-art results.

2. METHODOLOGY

This section describes the hybrid optimization algorithm used to optimize the Convolutional Neural Network (CNN) for the CIFAR-10 classification task. The hybrid algorithm combines the Genetic Algorithm (GA), Jellyfish Optimization (JFO), and Feedback Artificial Tree (FAT) to enhance the performance of the CNN by improving hyperparameter tuning, weight optimization, and feature selection.

2.1. Hybrid Optimization Algorithm

This is a hybrid algorithm that combines three different optimization techniques: Genetic Algorithm (GA), Jellyfish Optimization (JFO), and Feedback Artificial Tree (FAT). Each of these algorithms is significant in improving the performance of CNN by optimizing the hyperparameters, weights, and network architecture.

2.1.1 Genetic Algorithm (GA) for Hyperparameter Optimization

2.1.1.1 Initialization of Population

GA begins by initializing a population of possible solutions (individuals) representing different sets of hyperparameters for the CNN model.

Each individual Pi in the population is represented as a vector of hyperparameters:

$$P_i = \{\alpha_i, b_i, e_{i,\ldots}\} - (1)$$

Where:

α_i = learning rate,

b_i = batch size,

e_i = number of epochs, etc.

The population P consists of N individuals:

$$P = \{p1, p2, p3, \ldots, pN\} - (2)$$

2.1.1.2 Fitness Evaluation

Each individual pi is evaluated based on the fitness function. The fitness function evaluates how well the CNN performs for a given set of hyperparameters. This is typically based on the accuracy of the CNN on the validation set:

$$\text{Fitness } (p_i) = TP+TN/TP+TN+FP+FN - (3)$$

Where:

TP = True Positives

TN = True Negatives

FP = False Positives

FN = False Negatives.

This fitness function gives us a measure of how well the CNN is performing with the hyperparameters defined by p_i.

2.1.1.3 Selection

Selection is performed based on the fitness of each individual. The probability of an individual P_i being selected for reproduction is proportional to its fitness:

$$P(p_1) = \frac{Fitness(p_i)}{\sum_{j=1} Fitness(p_i)} - (4)$$

This ensures that individuals with better fitness have a higher chance of being selected for crossover.

2.1.1.4 Crossover

In the crossover phase, two selected parents p1and p2 combine their hyperparameters to create offspring p′. This is done by exchanging components of the hyperparameter vectors:

p' = Crossover (p1, p2)−(5)

For example, the learning rate α1 from p1might be combined with the batch size b2 from p2 and so on.

2.1.1.5 Mutation

To introduce genetic diversity, mutation is applied. Mutation involves random modifications to the offspring p′ with a small probability $p_{mutation}$. From equation 5 The mutation is represented as:

$$p'_{new} = P' + \epsilon \cdot \Delta_P - (6)$$

Where $\epsilon \sim U(0,1)$ is a random factor between 0 and 1, and Δp represents the random perturbation in the hyperparameter vector.

2.1.1.6 Output of GA (Hyperparameters to JFO)

Once GA converges, the optimal set of hyperparameters $P_{best} = \{\alpha_{best}, b_{best}, e_{best}\}$ is selected. These hyperparameters are passed as input to Jellyfish Optimization (JFO) for further refinement of the CNN weights:

GA Output = $p_{best} = \{\alpha_{best}, b_{best}, e_{best}\} - (7)$

2.1.2 Jellyfish Optimization (JFO) for Weight Optimization

Once GA provides the optimal hyperparameters, Jellyfish Optimization (JFO) takes these and optimizes the weights of the CNN model. The weights in the CNN model can be represented by $w_1, w_2, \ldots, w_N$.

2.1.2.1 Exploration

JFO starts with the exploration phase, where the jellyfish agents perform a random search through the solution space using a Levy flight strategy. The update equation for the position $x_i(t)$ of jellyfish i at iteration t is:

$$x_i(t+1) = x_i(t) + \alpha \cdot L(\lambda) - (8)$$

where:

$L(\lambda) \sim Levy\ Distribution,$

$x_i(t)\ is\ the\ position\ of\ jellyfish\ i\ at\ iteration\ t,$

$\alpha_{best}\ is\ the\ learning\ rate\ obtained\ from\ GA$

Here, $L(\lambda)$ represents a random jump (Levy flight) and α is a scaling factor.

This allows the algorithm to search broadly over the solution space.

2.1.2.2 Exploitation Phase (Best Solution)

After exploration, JFO moves into the exploitation phase, where the jellyfish agents focus on the best solution x_{best} found so far. The update equation for the position of jellyfish i becomes:

$$x_i(t+1) = x_i(t) + \beta \cdot (x_{best} - x_i(t)) - (9)$$

Where:

x_{best} *is the best solution found so far*

β *is the exploitation parameter that controls how strongly the agent are drawn towards* x_{best}

2.1.2.3 Output of JFO (Weights to FAT)

After optimizing the CNN weights, the best solution from JFO is passed to Feedback Artificial Tree (FAT) for further refinement. The output of JFO is the optimized weights:

JFO Output = $x_{best} = \{w_1, w_2, \dots, w_N\} - (10)$

Where w_i are the CNN weights optimized by JFO

2.1.3 Feedback Artificial Tree (FAT) for Adaptive Search

Finally, Feedback Artificial Tree (FAT) takes the output from JFO (the weights) and refines them by adjusting the balance between exploration and exploitation using feedback mechanisms.

2.1.3.1 Feedback Mechanism

In FAT, the feedback mechanism adjusts the search process based on the quality of the solutions found. The position update equation for solution $x_i$at iteration t is:

from equation (9).

$$x_i(t+1) = x_i(t) + \rho \cdot (x_{best} - x_i(t)) \cdot (1 - e^{-\alpha t}) - (11)$$

Where:

ρ *is a feedback factor,*

α *is a decay constant,*

t *is the current iteration.*

2.1.3.2 Solubility-Based Search

FAT also uses a solubility model to evaluate the fitness and stability of the solutions. The solubility A_i is updated using the following equation:

$$A_i = \lambda \cdot exp\left(-\frac{T_i}{T_{max}}\right) - (12)$$

Where:

T_i *is the solubility of the current solution,*

λ *is a constant,*

T_{max} *is the maximum solubility.*

2.1.3.3 Position Update in FAT

The final position update for solution in FAT is:

From equation (11) & (12).

$$x_i(t+1) = x_i(t) + A_i \cdot (x_{best} - x_i(t)) - (13)$$

Where:

A_i *adjusts the movement based on the solution's solubility and fitness.*

2.1.2.4 Final Output of FAT

The final output of FAT is the fully optimized weights of the CNN, which is ready for running:

FAT Output = $x_{final} = \{w_1, w_2, \dots, w_N\} - (14)$

Where:

w_i *are the final optimized weights.*

2.2 CNN Architecture

The CNN used for the CIFAR-10 classification consists of several layers designed to extract features and classify the images. The architecture is as follows:

Input Layer: Accepts 32x32 pixel RGB images, with a total of 1024 input neurons.

Convolutional Layers: Convolutional layers are used to learn spatial hierarchies in the image data. A typical convolution operation can be represented as:

$$Y(i,j) = \sum_{m=1}^{M} \sum_{n=1}^{N} x(i+m, j+n) \cdot k(m,n) - (15)$$

Where:

x *is the input image,*

k *is the kernel,*

$Y\ is\ the\ resulting\ feature\ map.$

Activation Function: The Rectified Linear Unit (ReLU) activation function is applied after each convolution:

$\text{ReLU}(x) = \max(0, x)$

Pooling Layers: Max-pooling layers reduce the spatial size of the feature maps. For a pool size of 2x2:

$Y(i,j) = \max(X(2i,2j), X(2i+1,2j), X(2i,2j+1), X(2i+1,2j+1))$

Fully Connected Layers: Flattened output from the last convolutional layer is passed to fully connected layers for classification:

$$Y_{fc} = w.X + b - (16)$$

Where:

$w\ is\ the\ weight\ matrix,$

$x\ is\ the\ falttened\ vector,$

$b\ is\ the\ bias.$

Output Layer: The output layer uses softmax activation to produce a probability distribution across the 10 classes:

$$p(y_i|x) = \frac{exp(w_i \cdot x + b_i)}{\sum_{j=1}^{10} exp\,(w_j \cdot x + b_j)} - (17)$$

Table 1: Details about the CNN Model

Layers	Filters	Filter Size	Feature Map	Parameters
Rescaling	-	-	32*32*3	0
Conv2D_1	32	3*3	32*32*32	896
MaxPooling 2D_1	-	2*2	16*16*32	0
Conv2D_2	64	3*3	16*16*64	18,496
MaxPooling 2D_2	-	2*2	8*8*64	0
Conv2D_3	128	3*3	8*8*128	73,856
MaxPooling 2D_3	-	2*2	4*4*128	0
Dropout_1	-	-	16*16*32	0
Dropout_2	-	-	8*8*64	0
Dropout_3	-	-	4*4*128	0
Flatten	-	-	2048	0
Dense_1	512	-	512	10,48,576

2.3 Optimization Process

The optimization process involves applying the Hybrid GA-JFO-FAT to the CNN:

Hyperparameter Optimization: The GA is used to optimize hyperparameters like learning rate, batch size, and number of layers.

Weight Optimization: JFO is used to fine-tune the CNN weights, and FAT ensures the search remains balanced between global exploration and local exploitation.

The overall process is iterative, with the hybrid optimization algorithm adjusting the CNN's parameters until a convergence criterion (e.g., a specified accuracy or loss threshold) is met.

2.4 Training and Evaluation

Training Procedure: The CIFAR-10 dataset is divided into training (50,000 images) and testing (10,000 images) sets. The CNN is trained using backpropagation and gradient descent with the Adam optimizer.

Evaluation Metrics: Performance is evaluated based on accuracy, precision, recall, and F1-score. The accuracy is computed as:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} - (18)$$

3. RESULTS AND DISCUSSIONS

Optimizing neural network performance is a core building block of advanced research and practical applications in the world of artificial intelligence and machine learning. With the rapid growth of data and models being highly complex, optimization techniques must be efficient in these times more than ever. In this paper, a hybrid optimization algorithm that incorporates the advantages of Genetic Algorithm (GA), Jellyfish Optimization (JFO), and Feedback Artificial Tree (FAT) to enhance the training and performance of a Convolutional Neural Network (CNN) on the CIFAR-10 dataset is proposed. The potential of hybrid approaches in modern deep learning pipelines is thus demonstrated by achieving a substantial accuracy improvement compared to traditional methods.

Table 2: Results of different models

Algorithm	Learning rate	Dropout rate	Accuracy(%)
Baseline Model (CNN)	0.001	0.3	79.9
CNN with jellyfish, FAT	0.00065561	0.3298038	82.91
CNN with GA, Jellyfish, FAT	0.000626336	0.22531089	84.11

The proposed hybrid optimization algorithm, combining Genetic Algorithm (GA), Jellyfish Optimization (JFO), and Feedback Artificial Tree (FAT), exhibited a

significant improvement in the performance of a Convolutional Neural Network (CNN) on the CIFAR-10 dataset. Baseline CNN without any optimization gives 79.9% accuracy. Adding JFO to optimize dropout rates and FAT for model refinement, accuracy goes up to 82.9%, thus a 2.9% improvement. Finally, adding GA for optimizing the learning rate brought an accuracy of 84.1%. This stepwise improvement will tell us about the effectiveness of hybrid approaches for improving learning capacity and generalization of the network.

The hybrid approach utilizes the benefit of each technique, where it accelerates GA convergence through optimized learning rate; JFO brings adaptive dropout towards regularization; FAT fine-tuned the model, making it more stable. Because of the fact that iterative optimization requires more computationally expensive operation, the benefit of this algorithm was proven at a 4.2% improvement over the baseline. Thus, these results reveal the synergy obtained by combining optimization methods in achieving different aspects in the training and present a potential avenue for improved deep learning performances on complex data sets like the CIFAR-10.

4. CONCLUSION

The integration of Genetic Algorithm (GA), Jellyfish Optimization (JFO), and Feedback Artificial Tree (FAT) into a hybrid optimization framework has proven to be a powerful approach for enhancing the performance of Convolutional Neural Networks (CNNs). By systematically optimizing key hyperparameters such as learning rate and dropout rate, and refining the model architecture, the hybrid algorithm achieved a notable accuracy of 84.1% on the CIFAR-10 dataset, outperforming the baseline by 4.1%. This demonstrates the potential of leveraging hybrid optimization strategies to overcome the limitations of standalone techniques, ensuring better generalization and robustness. The results highlight the importance of integrating evolutionary, swarm intelligence, and feedback-based approaches in pushing the boundaries of machine learning optimization toward their application in more complex real-world problems.

REFERENCES

[1] Krizhevsky, A., Sutskever, I., & Hinton, G. E. "ImageNet classification with deep convolutional neural networks." Communications of the ACM, vol. 60, no. 6, pp. 84-90, 2017. doi:10.1145/3065386.

[2] He, K., Zhang, X., Ren, S., & Sun, J. "Deep residual learning for image recognition." Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770-778. doi:10.1109/CVPR.2016.90.

[3] Simonyan, K., & Zisserman, A. "Very deep convolutional networks for large-scale image recognition." arXiv preprint arXiv:1409.1556, 2014.

[4] Hinton, G. E., Srivastava, N., & Krizhevsky, A. "Dropout: A simple way to prevent neural networks from overfitting." Journal of Machine Learning Research, vol. 15, pp. 1929-1958, 2014.

[5] Goodfellow, I., Bengio, Y., & Courville, A. "Deep Learning." MIT Press, 2016. ISBN: 9780262035613.

[6] Yegnanarayana, B. "Artificial Neural Networks." PHI Learning Pvt. Ltd., 2009. ISBN: 9788120321862.

[7] Kennedy, J., & Eberhart, R. "Particle swarm optimization." Proceedings of the IEEE International Conference on Neural Networks, vol. 4, 1995, pp. 1942-1948. doi:10.1109/ICNN.1995.488968.

[8] Mirjalili, S., Mirjalili, S. M., & Lewis, A. "Grey wolf optimizer." Advances in Engineering Software, vol. 69, pp. 46-61, 2014. doi:10.1016/j.advengsoft.2013.12.007.

[9] Heidari, A. A., Mirjalili, S., & Faris, H. "Harris hawks optimization: Algorithm and applications." Future Generation Computer Systems, vol. 97, pp. 849-872, 2019. doi:10.1016/j.future.2019.02.028.

[10] Karaboga, D., & Basturk, B. "A powerful and efficient algorithm for numerical function optimization: Artificial bee colony (ABC) algorithm." Journal of Global Optimization, vol. 39, no. 3, pp. 459-471, 2007. doi:10.1007/s10898-007-9149-x.

[11] Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. "A fast and elitist multiobjective genetic algorithm: NSGA-II." IEEE Transactions on Evolutionary Computation, vol. 6, no. 2, pp. 182-197, 2002. doi:10.1109/4235.996017.

[12] Geem, Z. W., Kim, J. H., & Loganathan, G. V. "A new heuristic optimization algorithm: Harmony search." Simulation, vol. 76, no. 2, pp. 60-68, 2001. doi:10.1177/003754970107600201.

[13] Rawat, W., & Wang, Z. "Deep convolutional neural networks for image classification: A comprehensive review." Neural Computation, vol. 29, no. 9, pp. 2352-2449, 2017. doi:10.1162/neco_a_00990.

[14] Tan, M., & Le, Q. V. "EfficientNet: Rethinking model scaling for convolutional neural networks." Proceedings of the 36th International Conference on Machine Learning, 2019, pp. 6105-6114.

[15] Rao, R. V., & Patel, V. K. "Multi-objective optimization of heat exchangers using Jaya algorithm." Applied Soft Computing, vol. 12, no. 5, pp. 2111-2121, 2012. doi:10.1016/j.asoc.2012.03.035.